

Automatiser les collectes avec GitHub Actions

Tutoriel pas à pas : sécuriser le compte et le dépôt, installer les quatre tâches planifiées, vérifier qu'elles publient réellement sur l'hébergement — et connaître les sept pièges qui font échouer ce genre de montage en silence.

Tout se fait depuis un navigateur, y compris sur téléphone. Aucune ligne de commande, aucun logiciel à installer.

1. Ce que l'on met en place — la chaîne en cinq maillons

2. Quatre vérifications préalables

3. Sécuriser le compte et le dépôt — cinq réglages

4. Le secret Météo-France

5. Le fichier de tâches planifiées

6. Premier essai, à la main

7. Vérifier que l'hébergement a bien reçu

8. Sept pièges connus

9. Sécurité en exploitation

10. Fiche de contrôle

11. En cas de problème

Document interne — 8 septembre 2026

À copier dans `Terri_Admin/Documents/`

1. Ce que l'on met en place

Aujourd'hui, vous lancez `python lancer.py` sur votre machine, puis vous envoyez les fichiers sur GitHub, et le webhook déploie chez OVH. Nous remplaçons seulement les deux premiers gestes.

GitHub Actions		le dépôt GitHub		OVH mutualisé
1. à l'heure dite	→	4. les fichiers	→	5. le webhook
2. exécute vos collecteurs		produits sont enregistrés dans le dépôt		déclenche le déploiement,
3. régénère le site				comme aujourd'hui

Les maillons 4 et 5 existent déjà et ne changent pas. Nous ajoutons les maillons 1 à 3, c'est-à-dire une machine louée quelques minutes par jour, qui fait ce que vous faites à la main.

Pourquoi pas le CRON d'OVH

L'hébergement mutualisé propose bien des tâches planifiées, mais elles exécutent du **PHP**, et l'interpréteur Python n'y est pas disponible. Vos seize collecteurs sont en Python. Réécrire en PHP reviendrait à jeter le travail fait et à maintenir deux langages. L'arbitrage est déjà inscrit au cahier des charges ; ce document en tire les conséquences.

Les quatre rythmes

Ils suivent la vitesse réelle de chaque source, pas une cadence uniforme. Interroger tous les jours une donnée annuelle ne servirait qu'à fatiguer des serveurs publics.

PLAN	QUAND	CE QU'IL COLLECTE	DURÉE
--quotidien	chaque jour, 5 h 17 UTC	restrictions sécheresse, vigilance météo	2 à 4 min
-- hebdomadaire	lundi, 5 h 37 UTC	niveau des nappes, débit des rivières	3 à 6 min
--mensuel	le 1 ^{er} , 5 h 47 UTC	eau potable, risques, écoles, équipements hivernaux	10 à 20 min
--trimestriel	5 janv., avr., juil., oct.	référentiel, population, élus, agriculture bio	15 à 30 min

Chaque plan se termine par l'agrégation puis la génération des pages : une collecte non publiée ne servirait à rien. Ces quatre plans sont définis dans `lancer.py`, et nulle part ailleurs — un fichier de tâche planifiée qui énumérerait lui-même les scripts finirait par diverger du lanceur sans que personne ne le voie.

POURQUOI LE TRIMESTRIEL EST À PART

Trois sources passent par de gros fichiers : 198 Mo pour l'archive INSEE, 78 Mo pour le répertoire des élus, 41 Mo pour l'Agence Bio. Les télécharger chaque mois n'aurait aucun sens — la population est annuelle, les élus trimestriels. Et il ne sert à rien d'espérer un cache : GitHub supprime tout cache inutilisé depuis sept jours, donc entre deux exécutions espacées il n'en resterait rien.

2. Quatre vérifications préalables

Dix minutes, et elles évitent de construire sur du sable.

À VÉRIFIER

où

- | | |
|---|---|
| 1 Le dépôt est privé. Il contient <code>Terri_Admin/.htpasswd</code> . Un dépôt public exposerait l'empreinte du mot de passe d'administration. | Page du dépôt : la mention « Private » figure à côté du nom |
| 2 Le webhook OVH fonctionne. C'est lui qui déploie. S'il est en panne, l'automatisation remplira le dépôt sans que le site bouge. | <code>Settings → Webhooks</code> : pastille verte, livraisons récentes en 200 |
| 3 Actions est activé. Sur un dépôt privé récent, l'onglet est présent par défaut. | Onglet <code>Actions</code> du dépôt |
| 4 Les seize scripts sont à jour dans le dépôt , y compris les fichiers de la dernière livraison. La machine de GitHub exécutera ce qui est dans le dépôt, pas ce qui est sur votre disque. | Onglet <code>Code</code> , date du dernier envoi |

LE PIÈGE DU POINT 4

C'est le plus fréquent de tous. Vous corrigez un script sur votre machine, vous ne l'envoyez pas, et la tâche planifiée continue de tourner avec l'ancienne version — sans erreur, avec des résultats faux. Le contrôle de versions du lanceur vous protège d'un fichier oublié *chez vous* ; il ne peut rien contre un fichier oublié *dans l'envoi*.

3. Sécuriser le compte et le dépôt — cinq réglages

À faire une fois, dans l'ordre. Tous depuis `Settings`, l'onglet du dépôt — pas celui du compte, sauf mention contraire.

3.1 La double authentification du compte

GitHub l'impose désormais à tous les comptes ; vérifiez qu'elle est bien active et que vos **codes de secours** sont conservés ailleurs que sur le téléphone qui sert à la validation. Un compte perdu, c'est le dépôt et le déploiement perdus avec lui.

`Compte → Settings → Password and authentication`

3.2 Les droits par défaut des tâches

Réglage le plus important de cette page. Par défaut, une tâche peut parfois écrire partout dans le dépôt. Nous voulons l'inverse : **aucun droit par défaut**, et une autorisation d'écriture accordée explicitement au seul fichier qui en a besoin.

Settings → Actions → General → Workflow permissions

- cochez **Read repository contents and packages permissions** ;
- décochez **Allow GitHub Actions to create and approve pull requests** ;
- enregistrez.

Notre fichier redemandera le droit d'écriture par une ligne `permissions: contents: write`. Si un jour un autre fichier de tâche apparaît sans cette ligne, il ne pourra rien modifier — c'est exactement ce que l'on veut.

3.3 Restreindre les actions autorisées

Une « action » est un bout de code écrit par un tiers que votre tâche exécute. C'est la principale surface d'attaque de ce montage : une action compromise s'exécute avec les droits que vous lui donnez, dans un dépôt qui contient votre `.htpasswd`.

Notre montage n'en utilise **qu'une seule**, publiée par GitHub lui-même : celle qui récupère le dépôt. Restreignez donc :

Settings → Actions → General → Actions permissions

- choisissez **Allow <votre compte>, and select non-<votre compte>, actions and reusable workflows** ;
- cochez **Allow actions created by GitHub** ;
- laissez décoché « Allow actions by Marketplace verified creators » ;
- enregistrez.

3.4 Être prévenu d'un échec

Une chaîne automatique qui tombe en panne sans prévenir est pire qu'une chaîne manuelle : le site vieillit en silence. GitHub envoie un courriel quand une tâche planifiée échoue ; vérifiez que c'est bien le cas.

Compte → Settings → Notifications → Actions — cochez l'envoi par courriel, et l'option qui limite l'envoi aux **seuls échecs**. Sans cette dernière, vous recevrez plus de trente courriels par mois et vous cesserez de les lire, ce qui revient à ne pas être prévenu.

3.5 Protéger la branche principale — facultatif

Sur un dépôt à un seul auteur, la protection de branche gêne plus qu'elle ne protège : la tâche planifiée écrit elle-même sur la branche principale. Si vous en mettez une un jour, il faudra autoriser explicitement le compte `github-actions[bot]` à la contourner. À laisser tel quel pour l'instant.

4. Le secret Météo-France

Un secret est une valeur que GitHub chiffre, expose à la tâche comme une variable d'environnement, et **masque automatiquement dans les journaux** : même si un script l'imprimait par erreur, elle s'afficherait sous forme d'astérisques.

Settings → Secrets and variables → Actions → New repository secret

CHAMP VALEUR

Name `METEOFRANCE_APIKEY` — exactement cette graphie, c'est le nom que `14_vigilance.py` lit dans l'environnement

Secret la clé obtenue sur `portail-api.meteofrance.fr`

Le collecteur cherche d'abord cette variable d'environnement, et seulement ensuite le fichier local `data/cle-meteofrance.json`. Rien à modifier dans le code : le mécanisme a été prévu pour cela.

TANT QUE LA CLÉ N'EST PAS DEMANDÉE

Vous pouvez tout installer sans elle. `14_vigilance.py` constate l'absence de configuration, l'explique, et rend la main sans interrompre la chaîne — un collecteur non configuré n'est pas une panne. La vigilance s'ajoutera d'elle-même le jour où vous créerez le secret.

5. Le fichier de tâches planifiées

Un seul fichier, quatre rythmes. Le créer depuis le navigateur :

Onglet Code → Add file → Create new file, puis saisir comme nom de fichier, en entier :

```
.github/workflows/collecte.yml
```

Les barres obliques créent les dossiers au fur et à mesure de la frappe. Collez ensuite le contenu ci-dessous, puis **Commit changes**.

```
name: Collecte et publication

on:
  schedule:
    - cron: '17 5 * * *'          # chaque jour
    - cron: '37 5 * * 1'         # chaque lundi
    - cron: '47 5 1 * *'         # le 1er du mois
    - cron: '7 6 5 1,4,7,10 *'   # chaque trimestre
  workflow_dispatch:
    inputs:
      plan:
        description: "Plan à exécuter"
        type: choice
        default: quotidien
        options: [quotidien, hebdomadaire, mensuel, trimestriel, site]
```

```

# Aucun droit par default, sauf celui d'ecrire dans le depot : c'est ce
# qui permet d'enregistrer les fichiers produits, et rien d'autre.
permissions:
  contents: write

# Deux executions ne doivent jamais publier en meme temps : la seconde
# serait refusee, et la collecte perdue. Elles font la queue.
concurrency:
  group: publication
  cancel-in-progress: false

jobs:
  collecte:
    runs-on: ubuntu-latest
    timeout-minutes: 60

    steps:
      - name: Recuperer le depot
        uses: actions/checkout@v4

      - name: Choisir le plan selon l'heure
        id: plan
        run: |
          case "${{ github.event.schedule }}" in
            "17 5 * * *")      plan=quotidien ;;
            "37 5 * * 1")      plan=hebdomadaire ;;
            "47 5 1 * *")      plan=mensuel ;;
            "7 6 5 1,4,7,10 *") plan=trimestriel ;;
            *)                  plan="${{ inputs.plan }}" ;;
          esac
          echo "plan=${plan:-quotidien}" >> "$GITHUB_OUTPUT"
          echo "Plan retenu : ${plan:-quotidien}"

      - name: Collecter et generer
        env:
          METEOFRANCE_APIKEY: ${ secrets.METEOFRANCE_APIKEY }
        run: python3 lancer.py --${{ steps.plan.outputs.plan }}

      - name: Publier dans le depot
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
          git add -A
          if git diff --cached --quiet; then
            echo "Aucun changement : rien a publier."
            exit 0
          fi
          git commit -m "Collecte ${ steps.plan.outputs.plan } du $(date -u '+%d/%m/%Y %H:%M
UTC')"
          git pull --rebase --autostash
          git push

      - name: Resume
        if: always()
        run: git log -1 --stat >> "$GITHUB_STEP_SUMMARY"

```

Ce que fait chaque bloc

BLOC	RÔLE
<code>schedule</code>	Les quatre rendez-vous. Les heures sont en UTC , jamais en heure locale — voir le piège n° 1. Les minutes ne sont pas rondes exprès : à l'heure pile, la file d'attente de GitHub est chargée.
<code>workflow_dispatch</code>	Ajoute un bouton Run workflow avec un menu déroulant. C'est lui qui sert au premier essai, et à toute exécution manuelle depuis le téléphone.
<code>permissions</code>	Le droit d'écrire dans le dépôt, accordé ici et seulement ici. Il ne donne accès ni aux réglages, ni aux secrets d'un autre dépôt.
<code>concurrency</code>	Le 1 ^{er} janvier, trois rendez-vous tombent presque ensemble. Sans cette précaution, deux exécutions publieraient en même temps et l'une serait rejetée.
<code>timeout-minutes</code>	Une collecte bloquée sur une API muette est arrêtée au bout d'une heure, plutôt que de consommer le quota jusqu'au bout.
Choisir le plan	GitHub indique quel rendez-vous a déclenché l'exécution ; cette étape le traduit en option du lanceur. Si vous modifiez une heure dans <code>schedule</code> , modifiez la même ligne ici — c'est le seul endroit où les deux doivent concorder.
Collecter et generer	Le lanceur, avec la clé Météo-France passée par l'environnement. S'il s'arrête sur un blocage, l'étape suivante n'est pas exécutée : rien n'est publié , et le site conserve ses données précédentes.
Publier dans le depot	Enregistre les fichiers produits. <code>git add -A</code> repose entièrement sur le <code>.gitignore</code> : c'est lui qui écarte les caches volumineux et la clé. D'où l'importance de la mise à jour livrée avec ce document.
Resume	Affiche la liste des fichiers modifiés sur la page de l'exécution. Sans cela, on ne sait pas ce qui a bougé — et ce qui n'est pas affiché n'est pas vérifié.

UNE SEULE ACTION TIERCE

`actions/checkout` est publiée par GitHub. Aucune autre n'est utilisée : pas d'installation de Python — la machine en dispose déjà — pas de cache, pas d'action de publication. Moins de code étranger exécuté, moins de surface d'attaque. Si un jour vous ajoutez une action trouvée sur le Marketplace, épinglez-la à une empreinte de version précise plutôt qu'à un numéro, et relisez ce qu'elle fait.

6. Premier essai, à la main

N'attendez pas 5 h 17 pour découvrir une erreur de frappe.

1. Onglet `Actions` → dans la colonne de gauche, **Collecte et publication**.
2. Bouton **Run workflow** à droite, plan **site** — il ne sollicite aucun serveur public et régénère seulement les pages. C'est l'essai le moins risqué.
3. Rafraîchissez : l'exécution apparaît au bout de quelques secondes. Ouvrez-la, déployez chaque étape.

Ce que vous devez lire dans le journal

ÉTAPE	ATTENDU
Choisir le plan	Plan retenu : site
Collecter et generer	Aucun message « fichiers non remplacés » ; chaque script annonce sa version ; aucun [BLOCAGE]
Collecter et generer	« Recherche : 47/47 communes joignables »
Publier dans le depot	soit un envoi, soit « Aucun changement : rien a publier » — les deux sont normaux
Resume	la liste des fichiers modifiés

UNE PARTICULARITÉ DU PLAN « SITE »

Sur la machine de GitHub, il retélécharge les contours communaux : le fichier `data/contours.json` est volumineux et exclu du dépôt. Comptez quelques minutes de plus qu'à la maison, et ne vous en inquiétez pas. Les quatre plans planifiés, eux, ne redessinent pas les cartes.

Puis recommencez avec le plan **quotidien** : celui-là interroge vraiment VigieEau et Météo-France. Un « rien a publier » est possible si les restrictions n'ont pas changé depuis votre dernière exécution.

7. Vérifier que l'hébergement a bien reçu

Une tâche verte ne prouve rien pour le visiteur : elle prouve que le dépôt a été mis à jour, pas que le site l'a été.

1. Settings → Webhooks → votre webhook OVH → onglet **Recent Deliveries**.
2. Une livraison doit porter l'horodatage de l'exécution, et répondre **200**.
3. Ouvrez le site et regardez la date de mise à jour en pied de page.

SI LE WEBHOOK NE SE DÉCLENCHÉ PAS

Les envois faits par une tâche planifiée utilisent un jeton technique. Les webhooks du dépôt s'en accommodent normalement. Si aucune livraison n'apparaît, la parade connue consiste à faire pousser la tâche avec une clé de déploiement plutôt qu'avec le jeton par défaut : l'envoi est alors attribué à un compte ordinaire. Signalez-le-moi, cela se règle en quelques lignes — mais vérifiez d'abord ce point, car il ne se voit pas autrement qu'ici.

8. Sept pièges connus

1 — Les heures sont en UTC, pas à votre montre

5 h 17 UTC, c'est 7 h 17 à Saint-Marcellin l'été et 6 h 17 l'hiver : le rendez-vous se décale d'une heure deux fois par an. Sans conséquence pour une collecte matinale, mais à savoir avant de vous étonner. La documentation mentionne désormais un champ facultatif de fuseau horaire ; tant que vous ne l'avez pas vu fonctionner sur votre dépôt, restez en UTC — c'est prévisible.

2 — Un rendez-vous n'est pas une garantie

GitHub prévient que les tâches planifiées peuvent être retardées en période de charge, parfois de plusieurs dizaines de minutes, et qu'un rendez-vous peut être purement et simplement sauté. Notre chaîne n'a aucune urgence : la sécheresse change une fois par jour. Le jour où les carburants arriveront, il faudra accepter qu'une relève manque de temps en temps — et c'est une raison de plus d'afficher l'horodatage plutôt que de le masquer.

3 — La désactivation après soixante jours

La documentation de GitHub prévoit la désactivation automatique des tâches planifiées après soixante jours sans activité sur le dépôt. **Cette règle est documentée pour les dépôts publics** ; le vôtre est privé, et vos tâches y écrivent tous les jours. Le risque est donc théorique. Il ne coûte rien de garder l'habitude de regarder l'onglet Actions une fois par mois : si les exécutions se sont arrêtées, un bouton les réactive.

4 — Le fichier oublié dans l'envoi

Déjà dit, et c'est le piège le plus coûteux : la machine exécute le contenu du dépôt. Après chaque livraison, envoyez les fichiers *puis* lancez une exécution manuelle : le contrôle de versions du lanceur s'exprimera dans le journal, et vous saurez tout de suite si un fichier manque.

5 — Le blocage silencieux

Un script qui rencontre une anomalie s'arrête sans rien écrire, et l'étape de publication ne s'exécute pas. Le site reste sur ses données précédentes — c'est voulu. Mais si personne ne lit le courriel d'échec, le site vieillit sans que rien ne le signale. D'où le réglage 3.4, et la page publique de fraîcheur des données inscrite à la feuille de route.

6 — Deux exécutions qui se marchent dessus

Traité par le bloc `concurrency`. Sans lui, l'exécution mensuelle du 1^{er} janvier et la trimestrielle du 5 auraient fini par se croiser un jour de rattrapage, et l'une des deux aurait été rejetée à l'envoi.

7 — Le quota

Un dépôt privé dispose d'un quota mensuel gratuit de minutes de calcul. Nos quatre plans consomment de l'ordre de deux heures par mois, très en deçà. Surveillez-le tout de même après la mise en service des carburants, qui demanderont plusieurs exécutions par jour :

Compte → Settings → Billing .

9. Sécurité en exploitation

Les réglages de la section 3 se font une fois. Ces règles-là valent tout le temps.

Ce qui ne doit jamais entrer dans un journal d'exécution

Les journaux d'un dépôt privé ne sont lisibles que par vous, mais ils sont conservés et exportables. Aucun script ne doit imprimer la clé Météo-France, le contenu de `.htpasswd`, ni une adresse complète de visiteur. Les collecteurs actuels ne le font pas. Si vous en ajoutez un, la règle tient en une phrase : **on affiche ce que l'on a compris de la donnée, jamais le moyen d'y accéder.**

Le dépôt reste privé

Tant qu'il contient `Terri_Admin/.htpasswd`, le rendre public publierait l'empreinte du mot de passe d'administration. Si vous voulez ouvrir le code un jour — ce qui aurait du sens pour un portail de données publiques — il faudra d'abord sortir ce fichier du dépôt et le déposer directement sur le serveur.

Rotation de la clé

Une clé d'API se remplace sans douleur : générez-en une nouvelle sur le portail Météo-France, modifiez le secret dans GitHub, lancez une exécution manuelle. Faites-le si la clé a transité par un courriel, un message, ou un document partagé.

Revue trimestrielle

Quatre points, dix minutes, à la même occasion que le passage du plan trimestriel :

- les exécutions des trois derniers mois sont-elles vertes ?
- le webhook OVH répond-il toujours 200 ?
- la liste des actions autorisées est-elle toujours restreinte ?
- `chiffrer.php` est-il toujours absent du dépôt ?

10. Fiche de contrôle

À dérouler une fois l'installation faite.

- ☐ Le dépôt est privé, et le webhook OVH répond 200.
- ☐ Les droits par défaut des tâches sont en lecture seule.

- ☐ Les actions autorisées sont limitées à celles de GitHub.
- ☐ Le secret `METEOFRANCE_APIKEY` existe, ou la clé n'est pas encore demandée et c'est assumé.
- ☐ Le courriel d'alerte en cas d'échec est activé, limité aux échecs.
- ☐ `.github/workflows/collecte.yml` est dans le dépôt.
- ☐ Une exécution manuelle en plan **site** se termine en vert.
- ☐ Une exécution manuelle en plan **quotidien** se termine en vert.
- ☐ Le journal ne signale aucun fichier en retard de version.
- ☐ Le résumé d'exécution liste bien des fichiers modifiés.
- ☐ Le webhook a livré, et la date de pied de page du site a changé.
- ☐ Le lendemain matin : une exécution automatique figure dans l'onglet Actions.

Les quatre derniers points sont les seuls qui prouvent que la chaîne fonctionne de bout en bout. Les autres ne prouvent que des intentions.

11. En cas de problème

SYMPTÔME	CAUSE LA PLUS PROBABLE	GESTE
« fichiers non remplacés » au démarrage	un script de la dernière livraison n'a pas été envoyé au dépôt	envoyez-le, relancez à la main
La tâche est verte mais le site ne change pas	le webhook OVH n'a pas livré	section 7, encadré
Rien ne se déclenche à l'heure dite	retard de file d'attente, ou heure UTC mal convertie	attendez une heure, puis relisez le piège n° 1
Échec à l'étape de publication	deux exécutions simultanées, ou envoi rejeté	relancez : le <code>git pull --rebase</code> règle le cas courant
Un collecteur s'arrête sur <code>[BLOCAGE]</code>	une source a changé de format, ou un contrôle a détecté une anomalie	c'est le mécanisme voulu : rien n'a été publié. Transmettez-moi la sortie complète du journal
« Aucune clé d'accès — rien à publier »	le secret Météo-France n'existe pas	normal tant que la clé n'est pas demandée ; section 4

CE QUE CE MONTAGE NE FAIT PAS

Il ne surveille pas le site, ne vérifie pas que les pages sont correctes, et ne détecte pas une donnée fausse qui passerait les contrôles. Il exécute votre chaîne, à l'heure, avec les mêmes garde-fous que sur votre machine. La page publique de fraîcheur des données, inscrite à la feuille de route, reste le complément qui manque : c'est elle qui rendra visible au visiteur — et à vous — qu'une source a cessé d'être mise à jour.

Sources vérifiées le 8 septembre 2026. Documentation GitHub — *Events that trigger workflows* (rendez-vous en UTC, intervalle minimal de cinq minutes, retards en période de charge, désactivation après soixante jours pour les dépôts publics) et *Disabling and enabling a workflow*. Les durées d'exécution annoncées sont des estimations tirées de vos propres relevés, non des mesures faites sur la machine de GitHub.